Python代码-07-引入精英策略

2024-06-03 修改每代选择最优个体的逻辑，引入精英策略，使每代最佳成本折线图的趋势稳定。整合生成折线图和共现矩阵的代码。

遗传派工算法5-0604 复制代码

```
import pandas as pd
import numpy as np
from deap import base, creator, tools, algorithms
import random
from sqlalchemy import create_engine
from collections import defaultdict
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator
import matplotlib.font_manager as fm
import seaborn as sns
from mpl_toolkits.axes_grid1.inset_locator import inset_axes
import copy

# MySQL 连接信息
mysql_connection_string =
'mysql+pymysql://ncc_01:ncc%21%40%23%24@211.138.201.6:3506/pcms_2022'
mysql_engine = create_engine(mysql_connection_string)

# 查询语句
query = """
SELECT *
FROM v_pp_weldlist
WHERE unitCode = '马来西亚项目'
AND (alterType IS NULL OR alterType NOT IN ('c', 'f', 'w'))
AND TRIM(remark) <> ''
"""
# 执行查询
df_mysql = pd.read_sql(query, mysql_engine)

# 关闭数据库连接
mysql_engine.dispose()

# 导出到 Excel
# excel_path = '数据表.xlsx'
# df_mysql.to_excel(excel_path, index=False)
```

```python
# 使用pandas的isna()函数来检查空值
df = df_mysql[pd.isna(df_mysql['finishDate']) |
(df_mysql['finishDate'].astype(str).str.strip() == '')].copy()

# 读取 Excel 文件，指定第3行为列名
df_welders = pd.read_excel('马来项目Shop焊工考试台账.xlsx', header=2)

# 修改第11列的列名为 'WPS No.'
df_welders.columns.values[10] = 'WPS No.'

# 填充 'Welder No.' 列的合并单元格内容
df_welders['Welder No.'] = df_welders['Welder No.'].ffill()

# 筛选 '焊接对象' 为 '管道'
df_welders_filtered = df_welders[df_welders['焊接对象'] == '管道']

# 提取 'Welder No.' 和 'WPS No.'
df_welders_filtered = df_welders_filtered[['Welder No.', 'WPS No.']]

# 读取 WPS 数据
df_wps = pd.read_excel('WPS.xlsx')

# 定义材料映射字典
material_mapping = {
    'A105': 'A106 Gr.B',
    'A106': 'A106 Gr.B',
    'A106-CE': 'A106 Gr.B',
    'API 5L': 'A106 Gr.B',
    'A53 Gr.B': 'A106 Gr.B',
    'A335 GR.P11': 'P11'
}

# 对于不在映射字典中的材料，保持原值
def map_material(material):
    if material in material_mapping:
        return material_mapping[material]
    else:
        return material

# 将df中的材料名称映射到WPS中使用的标准名称
df['material_standard'] = df['material'].apply(map_material)

# 解析壁厚范围，创建新列
df_wps[['min_thickness', 'max_thickness']] = df_wps['壁厚范围'].str.split('~',
expand=True).astype(float)
```

```python
# 初始化匹配的WPS列表列
df['matched_WPS'] = np.empty((len(df), 0)).tolist()

# 遍历每行数据，匹配WPS
for index, row in df.iterrows():
    matched_wps = df_wps[(df_wps['材质'] == row['material_standard']) &
                         (df_wps['min_thickness'] <= row['thickness']) &
                         (df_wps['max_thickness'] >= row['thickness'])]['PQR
No.'].tolist()
    df.at[index, 'matched_WPS'] = matched_wps

    # 如果没有找到匹配的WPS编号，打印警告
    if not matched_wps:
        print(f"警告：行 {index} 没有匹配到任何WPS编号。材料：
{row['material_standard']}，厚度：{row['thickness']}")

# 关联df_welders_filtered和df_wps来获取材质分类
df_welders_materials = df_welders_filtered.merge(df_wps[['PQR No.', '材质分
类']], left_on='WPS No.', right_on='PQR No.', how='left')

# 去重，确保每个焊工对每个PQR No.只保留一条记录
df_welders_materials = df_welders_materials.drop_duplicates(subset=['Welder
No.', 'PQR No.'])

# 按焊工号分组，获取每个焊工的材质分类列表
welder_materials = df_welders_materials.groupby('Welder No.')['材质分
类'].apply(set).reset_index()

# 计算每个焊工的日成本
def calculate_daily_cost(materials):
    if materials == {'碳钢'}:
        return 550
    elif materials == {'不锈钢'}:
        return 600
    elif materials == {'碳钢', '不锈钢'}:
        return 650
    else:
        return 550  # 默认成本，如果材质分类为空或其他情况

welder_materials['日成本'] = welder_materials['材质分
类'].apply(calculate_daily_cost)

# 合并回df_welders_filtered以包含日成本
```

```
df_welders_filtered = df_welders_filtered.merge(welder_materials[['Welder
No.', '日成本']], on='Welder No.', how='left')

# 创建适应度函数和个体类
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))  # 最小化适应度函数
creator.create("Individual", list, fitness=creator.FitnessMin)  # 基于适应度函数
的个体类

# 创建一个字典来存储每个WPS可以由哪些焊工执行
wps_to_welders = defaultdict(set)
for _, row in df_welders_filtered.iterrows():
    for wps in [row['WPS No.']]:  # 确保 WPS No. 正确处理
        wps_to_welders[wps].add(row['Welder No.'])

# 将集合转换为列表（如果需要在之后的处理中使用列表）
for wps in wps_to_welders:
    wps_to_welders[wps] = list(wps_to_welders[wps])

# 构建管线的应检测点口数量映射
line_to_inspection_spots = defaultdict(dict)
for line, group in df_mysql.groupby('lineCode'):
    for jtype, sub_group in group.groupby('jtype'):
        if 'rtRate' in sub_group.columns and not
sub_group['rtRate'].isnull().all():
            rt_rate = sub_group['rtRate'].dropna().astype(float).iloc[0] / 100
            if rt_rate > 0:
                spot_count = sub_group.shape[0]
                if spot_count > 0:
                    inspection_spots = np.ceil(spot_count * rt_rate)
                    line_to_inspection_spots[line][jtype] =
int(inspection_spots)
            else:
                line_to_inspection_spots[line][jtype] = 0
        else:
            line_to_inspection_spots[line][jtype] = 0

# 构建管线到已用焊工的映射
line_welders = defaultdict(lambda: defaultdict(set))
for line, welder, jtype in df_mysql[['lineCode', 'coverWelder',
'jtype']].dropna().values:
    line_welders[line][jtype].add(welder)

# 改进的初始化策略
def create_individual():
    individual = []
```

```
    for _, task in df.iterrows():
        line_code = task['lineCode']
        jtype = task['jtype']
        inspection_spots = line_to_inspection_spots.get(line_code,
{}).get(jtype, 0)

        # 统一处理 WPS 资质匹配的逻辑
        welders_pool = set()
        if isinstance(task['matched_WPS'], list):
            for wps in task['matched_WPS']:
                if wps in wps_to_welders:
                    welders_pool.update(wps_to_welders[wps])
        else:
            if task['matched_WPS'] in wps_to_welders:
                welders_pool.update(wps_to_welders[task['matched_WPS']])

        # 选择合适的焊工
        if welders_pool:
            if inspection_spots == 0 or inspection_spots == 100 or
inspection_spots is None:
                qualified_welders = list(welders_pool)
            else:
                if len(line_welders[line_code][jtype]) < inspection_spots:
                    qualified_welders = list(welders_pool)
                else:
                    # 如果已达到或超过inspection_spots，只能从已用焊工列表中选择
                    eligible_welders = line_welders[line_code]
[jtype].intersection(welders_pool)
                    if eligible_welders:
                        qualified_welders = list(eligible_welders)
                    else:
                        # 如果已用焊工列表中没有符合WPS的焊工，允许超限
                        qualified_welders = list(welders_pool)

            if qualified_welders:
                welder = random.choice(qualified_welders)
            else:
                welder = "NONE"
        else:
            welder = "NONE"

        individual.append(welder)
        # 更新line_welders列表
        line_welders[line_code][jtype].add(welder)
```

```python
    return creator.Individual(individual)

# 设置遗传算法的工具箱
toolbox = base.Toolbox()
toolbox.register("attr_welder", random.choice, df_welders_filtered['Welder No.'].unique().tolist())  # 随机选择一个焊工号作为属性
toolbox.register("individual", create_individual)  # 使用新的个体创建策略
toolbox.register("population", tools.initRepeat, list, toolbox.individual)  # 创建种群，种群由多个个体组成

# 评估函数
def evalWelding(individual):
    total_days = 0
    total_penalty = 0
    total_cost = 0  # 总成本初始化
    tasks_remaining = df.copy()
    task_days = {}  # 记录任务被安排的天数
    daily_summaries = []  # 新增一个列表来存储每天的摘要信息
    task_details = {}  # 新建字典来记录每个任务的详细信息

    # 初始化welders_per_line包含line_welders中的数据
    welders_per_line = defaultdict(lambda: defaultdict(set))
    for line, jtype_welders in line_welders.items():
        for jtype, welders in jtype_welders.items():
            welders_per_line[line][jtype].update(welders)

    while not tasks_remaining.empty:
        daily_cost = 0  # 每天的成本初始化
        daily_workloads = {}
        tasks_to_drop = []
        welders_used_today = set()  # 记录当天使用的焊工

        for i, (index, task) in enumerate(tasks_remaining.iterrows()):
            welder = individual[i]

            # 确保焊工具备所需的WPS资格
            welders_pool = set()
            if isinstance(task['matched_WPS'], list):
                for wps in task['matched_WPS']:
                    if wps in wps_to_welders:
                        welders_pool.update(wps_to_welders[wps])
            else:
                if task['matched_WPS'] in wps_to_welders:
                    welders_pool.update(wps_to_welders[task['matched_WPS']])
```

```
            line_code = task['lineCode']
            jtype = task['jtype']
            inspection_spots = line_to_inspection_spots.get(line_code,
{}).get(jtype, 0)

            # 如果焊工不具备资格或已达到inspection_spots，选择一个符合要求的焊工
            if welder not in welders_pool:
                if len(welders_per_line[line_code][jtype]) < inspection_spots:
                    if welders_pool:
                        welder = random.choice(list(welders_pool))
                    else:
                        welder = "NONE"
                else:
                    eligible_welders = welders_per_line[line_code]
[jtype].intersection(welders_pool)
                    if eligible_welders:
                        welder = random.choice(list(eligible_welders))
                    else:
                        if welders_pool:
                            welder = random.choice(list(welders_pool))
                        else:
                            welder = "NONE"

            # 如果已达到inspection_spots
            if len(welders_per_line[line_code][jtype]) >= inspection_spots:
                eligible_welders = welders_per_line[line_code]
[jtype].intersection(welders_pool)
                if eligible_welders:
                    welder = random.choice(list(eligible_welders))
                else:
                    if welders_pool:
                        welder = random.choice(list(welders_pool))
                    else:
                        welder = "NONE"

            if welder == "NONE":
                continue  # 如果没有符合要求的焊工，跳过此任务

            if welder not in daily_workloads:
                daily_workloads[welder] = 0

            if daily_workloads[welder] + task['jsize'] <= 50:
                daily_workloads[welder] += task['jsize']
                tasks_to_drop.append(index)
                task_days[index] = total_days + 1  # 记录任务的安排天数
```

```
                welders_used_today.add(welder)
                task_details[index] = {'welder': welder, 'day': total_days + 1}  # 记录任务的详细信息
                welders_per_line[line_code][jtype].add(welder)  # 更新焊工分配记录

        daily_total = sum(daily_workloads.values())
        daily_cost = sum(df_welders_filtered.loc[df_welders_filtered['Welder No.'] == welder, '日成本'].iloc[0] for welder in welders_used_today)  # 计算当天的总成本
        tasks_remaining.drop(tasks_to_drop, inplace=True)
        daily_summaries.append(f"第 {total_days + 1} 天: 完成 {daily_total} 寸, 剩余总寸径量 {tasks_remaining['jsize'].sum()}, 当天焊工人数 {len(welders_used_today)}, 当天焊工成本 {daily_cost}")

        # 惩罚1: 每天焊接量不足1500寸径量
        if daily_total < 1500:
            total_penalty += 100  # 不足1500寸径量的惩罚

        # 惩罚2: 每天每有一个焊工没有安排工作
        total_penalty += (len(df_welders_filtered['Welder No.'].unique()) - len(welders_used_today)) * 5

        # 惩罚3: 每个派出的焊工每天工作量不足20寸径量
        for welder, workload in daily_workloads.items():
            if workload < 20:
                total_penalty += 5  # 每个工作不饱和的焊工惩罚

        total_cost += daily_cost  # 累加每天的成本到总成本
        total_days += 1
        print(f"第 {total_days} 天: 完成 {daily_total} 寸, 剩余总寸径量 {tasks_remaining['jsize'].sum()}, 当天焊工人数 {len(welders_used_today)}, 当天焊工成本 {daily_cost}")

        if not tasks_to_drop:  # 如果一天没有任务被分配
            total_penalty += 1000  # 增加高额惩罚以避免无限循环
            break

    print(f"个体评估: 总天数 = {total_days}, 总惩罚值 = {total_penalty}, 总焊工成本 = {total_cost}\n")  # 打印每个个体的总天数和总惩罚值和总焊工成本
    return total_days, total_penalty, task_days, daily_summaries, total_cost, task_details  # 返回总天数和任务天数映射 # 返回每个任务的详细信息

# 注册遗传算法的操作
```

```
toolbox.register("evaluate", evalWelding)  # 注册评估函数，确保它只返回适应度相关的一个值
toolbox.register("mate", tools.cxTwoPoint)  # 注册交叉操作
toolbox.register("mutate", tools.mutShuffleIndexes, indpb=0.2)  # 注册变异操作
toolbox.register("select", tools.selTournament, tournsize=3)  # 注册选择操作

# 初始化一个列表来存储每一代的最优个体成本
best_costs_per_gen = []

# 初始化种群
population = toolbox.population(n=50)
NGEN = 50
elite_size = 2
best_costs_per_gen = []

for gen in range(NGEN):
    print(f"第 {gen + 1} 代: ")
    # 选择精英个体，并使用深拷贝保持其状态
    elite = tools.selBest(population, elite_size)
    elite_copies = copy.deepcopy(elite)

    # 生成后代，不包括精英个体
    offspring = algorithms.varAnd(population, toolbox, cxpb=0.8, mutpb=0.2)
    selected_offspring = toolbox.select(offspring, len(population) - elite_size)

    # 评估后代的适应度，注意避免在精英个体上调用评估函数
    for ind in selected_offspring:
        total_days, total_penalty, task_days, daily_summaries, total_cost, task_details = toolbox.evaluate(ind)
        # 组合多个值作为适应度
        weighted_fitness = 10000 * total_days + 10 * total_penalty + total_cost
        ind.fitness.values = (weighted_fitness,)  # 设置适应度
        ind.extra_info = (total_days, total_penalty, task_days, daily_summaries, total_cost, task_details)

    # 添加未修改的精英副本回种群
    population[:] = selected_offspring + elite_copies

    # 更新精英个体的适应度和信息，使用深拷贝后的副本进行评估
    for ind_copy, ind_original in zip(elite_copies, elite):
        total_days, total_penalty, task_days, daily_summaries, total_cost, task_details = toolbox.evaluate(ind_copy)
        weighted_fitness = 10000 * total_days + 10 * total_penalty + total_cost
        ind_original.fitness.values = (weighted_fitness,)
```

```
        ind_original.extra_info = (total_days, total_penalty, task_days,
daily_summaries, total_cost, task_details)

    # 记录并打印当前代的最佳成本和对应的精英
    best_ind_gen = tools.selBest(population, 1)[0]
    best_costs_per_gen.append(best_ind_gen.extra_info[4])
    print(f"当前最佳成本: {best_ind_gen.extra_info[4]}\n")

# 输出最终最佳解的详细信息，无需重新评估
best_ind = tools.selBest(population, 1)[0]
total_days, total_penalty, task_days, best_daily_summaries, total_cost,
best_task_details = best_ind.extra_info

print("最优解的每日任务摘要: ")
for summary in best_daily_summaries:
    print(summary)
print(f"个体评估: 总天数 = {total_days}, 总惩罚值 = {total_penalty}, 总焊工成本 =
{total_cost}\n")

# 更新DataFrame
df['Assigned Welder'] = df.index.map(lambda x: best_task_details.get(x,
{'welder': 'NONE'})['welder'])
df['Day Assigned'] = df.index.map(lambda x: best_task_details.get(x, {'day':
'NONE'})['day'])

# 输出和保存结果
output_excel_path = 'Updated_Data.xlsx'
df.to_excel(output_excel_path, index=False)
print(f"结果已保存到 {output_excel_path}")

# 设置中文字体，确保系统中安装了相应的中文字体，如SimHei或SimSun
plt.rcParams['font.sans-serif'] = ['SimHei']  # 使用黑体
plt.rcParams['axes.unicode_minus'] = False  # 解决保存图像时负号显示为方块的问题

# 创建x轴标签
x_labels = list(range(1, NGEN + 1))

# 生成折线图
plt.figure(figsize=(10, 6))
plt.plot(range(1, NGEN + 1), best_costs_per_gen, marker='o', linestyle='-',
color='b')

# 在每个数据点旁边标注成本数值
for gen, cost in enumerate(best_costs_per_gen, start=1):
    plt.text(gen, cost, f'{cost}', ha='center', va='bottom')
```

```python
plt.xlabel('代数')
plt.ylabel('总成本')
plt.title('总成本迭代折线图')

# 设置x轴刻度为整数
plt.gca().xaxis.set_major_locator(MultipleLocator(1))
plt.xticks(ticks=range(1, NGEN + 1), labels=x_labels)

plt.grid(True)
plt.savefig('总成本迭代折线图.png')  # 保存折线图

# 显示图片
# plt.show()

# 生成共现矩阵图
# 映射 'areaCode' 列的值
area_code_map = {'900-01': '900', '900-02': '900', '900-03': '900', '900-04':
'900', '900-05': '900'}
df['areaCode'] = df['areaCode'].replace(area_code_map)

# 创建一个透视表来计算共现矩阵
co_occurrence_matrix = df.pivot_table(index='areaCode', columns='Day
Assigned', values='Assigned Welder', aggfunc=lambda x: len(set(x)))

# 重命名行和列的名称
co_occurrence_matrix.index.name = '单元'
co_occurrence_matrix.columns.name = '天数'

# 计算每天派出的焊工总数量
daily_welders_total = df.groupby('Day Assigned')['Assigned Welder'].nunique()

# 计算当天安排的工作量
daily_workload = df.groupby('Day Assigned')['jsize'].sum()

# 计算剩余总工作量
total_jsize = df['jsize'].sum()
cumulative_workload = daily_workload.cumsum()
remaining_workload = total_jsize - cumulative_workload

# 合并这些信息到一个 DataFrame
summary_df = pd.DataFrame({
    '焊工总数量': daily_welders_total,
    '当天工作量': daily_workload,
    '剩余工作量': remaining_workload
```

南京南化建设有限公司 夏斐 | 5299

```
})

# 从df_welders_filtered中提取焊工成本信息
welder_costs = df_welders_filtered[['Welder No.', '日成
本']].drop_duplicates().set_index('Welder No.')

# 计算每天的焊工总成本，确保每个焊工只计算一次
daily_welder_costs = df.groupby('Day Assigned')['Assigned
Welder'].apply(lambda welders: sum(welder_costs.loc[w, '日成本'] for w in
set(welders) if w in welder_costs.index))

# 更新summary_df DataFrame以包含每天的总成本
summary_df['焊工总成本'] = daily_welder_costs

# 计算成本汇总
cumulative_cost = daily_welder_costs.cumsum()

# 创建图形和主轴
fig, ax = plt.subplots(figsize=(20, 15))

# 绘制热图
sns.heatmap(co_occurrence_matrix, annot=True, cmap='Blues', cbar=True,
linewidths=.5, yticklabels=1, xticklabels=1, ax=ax)
plt.yticks(rotation=0)  # 将纵轴标签设置为水平显示
plt.title('各单元每日派工数量共现矩阵')

# 添加第二个轴用于显示统计信息，位置在主轴下方，但更靠近 X 轴
ax2 = inset_axes(ax, width="100%", height="5%", loc='lower center',
bbox_to_anchor=(0, -0.05, 1, 1),
                 bbox_transform=ax.transAxes, borderpad=0)
ax2.set_xlim(ax.get_xlim())
ax2.set_xticks(ax.get_xticks())
ax2.set_xticklabels([f'焊工数量: {daily_welders_total.get(day, 0)}人\n工作量:
{daily_workload.get(day, 0)}寸\n剩余量: {remaining_workload.get(day, 0)}寸\n当日
成本: {daily_welder_costs.get(day, 0)}元\n汇总成本: {cumulative_cost.get(day,
0)}元'
                    for day in co_occurrence_matrix.columns], ha='center',
fontsize=5)  # 调小字号
ax2.tick_params(axis='x', rotation= 0)  # 取消标签旋转

# 隐藏 ax2 的 Y 轴和边框，因为它只用于显示 X 轴标签
ax2.set_frame_on(False)
ax2.get_yaxis().set_visible(False)

# 保存图片，设置高清晰度
```

南京南化建设有限公司 夏斐 | 5299